

营销活动

优惠券 / 秒杀 / 抢红包——业务取舍、ROI 与公平性

gomall · 业务侧 deck

今日大纲

营销在 gomall 业务版图里的位置

优惠券：促单工具 + ROI 计算

秒杀：亏本引流 + 流量峰值

抢红包：公平性是用户体验生死线

三个活动的横向业务对比

黑产对抗：业务后果与防线分层

客诉处理：从业务码到客服话术

运营 SOP：活动准备 + 盘点

各角色看到的关键指标

Q&A

营销在 gomall 业务版图里的位置

为什么营销是一个独立 deck：业务视角的三条线

- gomall 主链路是下单 → 支付 → 履约，营销是并行链路：在用户开始下单决策之前动手。
- 三条业务线，目的截然不同：
 - 优惠券：促单——把“看了不下单”的犹豫用户推过决策线。
 - 秒杀：引流——用“亏本商品”换 UV / 新客 / APP 打开率。
 - 抢红包：留存——让老用户每天有理由回来打开 APP。
- 三条线共享一套底层武器：**Redis Lua 原子、滑动窗口反作弊、Saga 兜底**。
- 本 deck 关心两件事：(1) 每条线的业务取舍是什么 (2) 用同一套武器为什么能打三场不同的仗。

internal/coupon/service.go, internal/redpacket/service.go, internal/skill/service.go 三个 service 入口

营销活动的 5 角色视图

角色	营销活动对他意味着什么	他最怕看到什么
C 端用户	”抢券、秒杀、抽红包”的兴奋时刻	抢券失败 / 红包金额很少 / 秒杀页面卡死
商家	我的活动能引流多少？带来多少新客？	平台羊毛党把券都领走，真实用户没拿到
运营	预算 → GMV 的 ROI 算不算得过来	100 万发券，只换来 5 万 GMV 增量 ($ROI < 1$)
客服	大促当天投诉量翻 5 倍，话术够不够用	用户问”为什么我抢不到红包”，没有标准回复
SRE	流量峰值是平常 50x，集群扛得住吗	0 点限流误伤真用户 / Redis Lua 超时

核心矛盾：同一个活动，5 个角色的成功定义完全不同——运营要”参与率高”，SRE 要”流量平稳”，客服要”话术清晰”，商家要”真客户拉到”，C 端用户要”抢得到 + 公平”。本 deck 的每一帧都会回答：”这个设计满足了谁，牺牲了谁。”

README 业务全景 5 角色表 + SLO P2 营销秒杀等级

业务承诺 SLO：营销是 P2，但峰值远超平日

活动	业务等级	可用率	p99 延迟	P2 不是次要：
优惠券领取	P2 营销	99%	< 200ms	
秒杀下单	P2 营销	99%	< 200ms	
抢红包	P2 营销	99%	< 200ms（资金敏感建议 < 300ms）	

用户的购物决策在大促当天集中爆发，营销失败意味着“用户走了，整年的 GMV 都拉不回”。
流量画像：平日营销接口 100 QPS，大促 0 点 5-10 万 QPS (50-100x)。

- 宁可下游慢、不能上游 429：营销可以排队、可以“已抢完”，但不能 5xx 让用户看到崩溃。
- 99% 可用率的潜台词：可以接受全场 1% 用户被限流误伤，但不接受 1% 用户超发（资损）。
- P2 的代价：实测压测 30VU × 15s 通过 46 / 期望 45 / 误差 2.2% ——误伤 < 5% 才算合格。

业务边界：这份 deck 覆盖什么、不覆盖什么

优惠券领取

秒杀抢购

抢红包

AB 测试

用户分群

满减引擎 (deck 13)

会员等级

积分体系

拼团 (deck 14)

做了的（绿）：三个核心 Lua 原子机

制 + Saga 兜底 + 限流防黑产 + 客服话术映射；满减引擎与拼团已另行落地，各有专属 deck (13 / 14)，本 deck 不展开。

没做的（红）：**AB 测试** / 用户分群 / 商家自助活动配置 / 积分 / 会员等级 / 直播带货。

为什么不做：这些是运营 / 商家工作台的事，gomall 当前阶段聚焦”交易闭环 + 防资损”。

等 wallet 与 merchant 服务落地后再做。README ”业务边界（不做什么）”段；

<docs/architecture/feature-matrix.md>

优惠券：促单工具 + ROI 计算

优惠券的业务目的：把犹豫用户推过决策线

- **业务公式**：商品定价 100 元，用户犹豫 → 发 20 元券 → 净付 80 元 → 下单。平台损失 20 元，但换来一笔本来不会发生的订单 (GMV +80)。
- **触达分层**：
 - **新客券**：注册赠送 50 元，目的是**首单转化**（行业 LTV 计算依赖首单完成）。
 - **老客回流券**：90 天未下单触发，目的**召回沉睡用户**。
 - **大促预热券**：提前 7 天发，让用户”提前决策”，0 点不挤兑结算系统。
- **为什么必须”先领后用”**：让用户产生”沉没成本错觉”——已经领了的券不用就亏了。
- **gomall 现状**：只做最基础的”批次 + 单人限领”，没做触达分层（运营平台路线图）。

coupon.CouponBatch.Type/Threshold/Amount 字段；internal/coupon/service.go:27-54 CreateBatch

活动 ROI 业务化帧：100 万发券能换多少 GMV

运营视角：一次活动是不是值，看这张表。

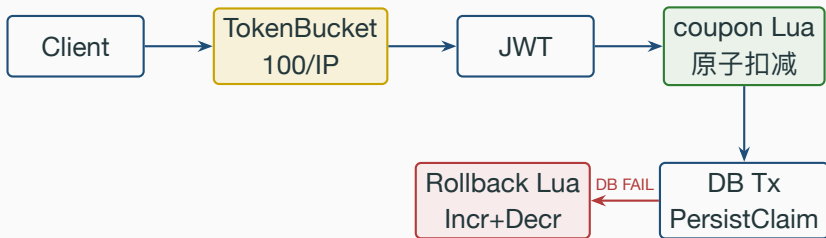
环节	行业基准	算式	数值
发券量	50000 张 (满 100 减 20)	单券面值 20 元	预算 100 万
领取率	60% (券进了用户卡包)	$50000 \times 60\%$	30000 张已领
核销率	20% (实际下单使用)	$30000 \times 20\%$	6000 张已用
带动 GMV	用券单平均客单 150 元	6000×150	90 万
平台让利	实际抵扣金额	6000×20	12 万
拉新效应	新客占用券 30%	$6000 \times 30\%$	1800 新客
ROI 计算	GMV / 让利	90 万 / 12 万	7.5

运营怎么判合格：核销率 20%

& ROI > 5 算合格。低于 10% 核销率 = 券面值太低/门槛太高，用户懒得用。

被黑产吃掉的代价：如果 30% 券进了脚本党的手里（领了不用 / 倒卖），**真实拉新打 7 折，核销率打 7 折，整个 ROI 直接腰斩**。这就是下文限流的钱字号意义。*README 业务承诺 + internal/coupon/service.go Claim 流程；ROI 算式为行业常识、gomall 未实装*

coupon 接口的中间件链：业务诉求决定每一层



业务取舍：coupon 链路故意做轻，只挂全局 TokenBucket。原因——

- 业务侧**鼓励多领**：用户每多领一张就多一份”沉没成本”，转化率正相关。
- Lua 里已经管控了 **PerUser 单人上限**，业务码 -2 直接挡。
- 单独挂 SlidingWindow 反而限制活跃用户（频繁刷新页面也会被截）。

代价：万号脚本党可以慢慢扫。靠 IP TokenBucket + 风控（路线图）兜底。*routes/router.go:41*

全局桶；*internal/coupon/routes.go:10-13* coupon 路由（无 SlidingWindow）

coupon Lua: 库存 + 单人限领二维原子

业务约束：两道闸必须同时锁——先 DECR 再判单人 = 已超限请求会留”幽灵库存”。代码顺序是有讲究的。

Listing 1: *repository/cache/coupon.go:30–44 claimScript*

```
30 local stock = tonumber(redis.call('GET', KEYS[1]))
31 if stock == nil or stock <= 0 then
32     return -1 -- 已抢光
33 end
34 local owned = tonumber(redis.call('GET', KEYS[2]))
35 if owned == nil then owned = 0 end
36 if owned >= tonumber(ARGV[1]) then
37     return -2 -- 超出单人上限
38 end
39 redis.call('DECR', KEYS[1])
40 redis.call('INCR', KEYS[2])
41 redis.call('EXPIRE', KEYS[2], tonumber(ARGV[2]))
42 return 1
```

业务码映射：1 → 200 成功；-1 → " 已抢光" 终态；-2 → " 每人限领 N 张"。Lua 返回 int，Go 侧 switch

coupon 回滚必须原子：两条独立命令留下的资损口子

业务场景： Lua 已扣库存 + 记单人，但下一步 DB PersistClaim 落库失败，必须把这两个计数一起还回去。早期回滚用两条独立命令、且吞掉错误：

Listing 2: 旧实现：库存与已领标记分两次写、错误被丢弃

```
1 RedisClient.Incr(ctx, stockKey) // 还库存
2 RedisClient.Decr(ctx, userClaimedKey) // 撤单人标记 —— 两条独立、无返回
```

业务后果： 两条命令之间崩溃或第二条失败，会出现半还状态——

- 库存还了、单人标记没撤：用户白白少领一张（明明没领到却被记了一次）。
- 单人标记撤了、库存没还：这一张库存幽灵消耗，别人永远抢不到。
- 错误被吞 → 运营无从察觉，活动结束后盘点才发现发券数对不上。

repository/cache/coupon.go:87-93 RollbackCouponStock

修法：单条 Lua 原子归还 + 返回 error 供补偿

Listing 3: *repository/cache/coupon.go:50–54 rollbackScript*

```
50 redis.call('INCR', KEYS[1]) -- 归还库存
51 redis.call('DECR', KEYS[2]) -- 撤销用户已领标记
52 return 1
```

两个动作放进同一个 **Lua** 单元，Redis 单线程执行保证同进同退，不再有半还状态。这与领券 `claimScript` 是同一套纪律——扣减原子，回滚也必须原子。

- **返回 error 不再静默**：RollbackCouponStock 把脚本错误向上抛；业务层回滚失败时打 `Errorf` 级告警，带上 `userId / batchId` 便于人工补偿对账。
- **对账闭环**：库存计数与单人标记永远一致 → 盘点时**发券数 = DB 行数 = Redis 消耗三者对得上**。

repository/cache/coupon.go:89–93 返回 error; internal/coupon/service.go:91–94 回滚失败 Errorf 告警

coupon 压测数字：业务侧怎么解读这两组数据

	Redis Lua	DB FOR UPDATE
500 并发抢 100 张	零超发	零超发
吞吐 RPS	51,362	50,142
avg 延迟	0.82ms	0.83ms
max 延迟	136ms	453ms

业务侧解读：吞吐拉平不等于体验一样。

- 尾延迟差 **3.3x**：DB 锁路径下，500 个用户里有几个会等 0.45s。

- 用户感知：DB 路径 = ”刷新两次才出结果”。

- 大促日叠加重试 = 雪崩。

结论：吞吐看 avg，体验看 p99 / max。

零超发是底线：1000 goroutine 抢 100 张券 → 成功数恰好 100，DB 行数 100，Redis 库存归 0，三者一致。

业务侧再次确认：这个不是技术指标，这是”不被资损”的底线——多发出去一张券就是真金白银的损失。

踩坑回填：DB 路径的 FOR UPDATE 曾因裸拼 SQL 退化成无锁空操作（并发下照样超发），现已改用 gorm clause.Locking 真实行锁；券码也补了 crypto/rand 随机段，同一用户同一秒连领不再撞唯一索引。*stressTest/REPORT.md* 链路压测表第 5-6 行；

repository/cache/coupon.go:30-44; internal/coupon/repo.go:44-90 ClaimWithDBLock、:127-137

generateCouponCode

秒杀：亏本引流 + 流量峰值

秒杀的业务本质：用”亏本商品”换”全场曝光”

- **业务公式：**iPhone 标价 8000 元，秒杀价 999 元 → 100 台 → 平台亏 **70 万**。但带来全场 **500 万 UV + APP 当日打开率 +30% + 大量”陪跑”用户顺手买别的**。
- **ROI 拆解：**
 - 直接亏损：70 万（秒杀差价）
 - 间接 GMV：500 万 UV × 0.5% 陪跑转化 × 客单 200 = **500 万**
 - 新增日活：APP 当日 DAU +30%（次日留存 30-40%，长期 LTV 收益）
- **业务取舍：**秒杀必须有”抢不到”的人。如果 100 台 iPhone 5 分钟才卖完，那不叫秒杀。用户预期是”瞬间售罄”，没抢到不是产品问题，是运气问题。
- **对应客服话术：**从来不说”系统繁忙”，永远说”已被其他用户抢走，欢迎下次参与”。

internal/skill/service.go:144-152 GetSkillGoods / SkillProduct; 商品独立 skill.SkillProduct 表

秒杀详情/下单读了一个从没写过的 key：接口必然失败

业务现象：秒杀详情与下单接口**每次都报错**，用户点开页面就是空白。根因是读写 **key** 对不上——预热只写了**列表 key**，详情/下单却按 `product_id` 读**单品 key**，而这个单品 key 从**没被写入**。

Listing 4: 读单品 key：缓存里永远是 `redis.Nil`

```
77 key := fmt.Sprintf(cache.SkillProductKey, productId) // skill:product:<id>  
78 resp, err := rc.Get(ctx, key).Result() // 旧代码到此必空
```

业务后果：秒杀是引流主入口，详情打不开 = 用户连“抢不到”的机会都没有 = 活动当场作废，比“手慢没抢到”严重得多。*repository/cache/key.go:12 SkillProductKey;*
internal/skill/service.go:75-85 loadSkillProduct 读取点

修法：预热补写单品 key + 读侧未命中回源回填

双管齐下，不再依赖单一预热顺序：

- 写侧：InitSkillGoods 在 LPush 列表的同时，按 product_id 多写一份**单品 key**（带 30min TTL）。
- 读侧：详情/下单未命中时**回源 DB** GetByProductId 并回填缓存，彻底摆脱”必须先预热”的隐式依赖。

Listing 5: *internal/skill/service.go:61-63* 预热补写单品 key

```
61 errx = cache.RedisClient.Set(ctx,  
62     fmt.Sprintf(cache.SkillProductKey, spList[i].ProductId),  
63     jsonString, cache.SkillProductTTL).Err()
```

读侧回填见 loadSkillProduct：Get 命中直接返回；redis.Nil 则查 DB、回写同一 key 再返回；其余错误才向上抛。GetSkillGoods（详情）与 SkillProduct（下单）共用这条读路径。*internal/skill/service.go:75-103*
loadSkillProduct 回源回填；*internal/skill/repo.go:40-45* GetByProductId

秒杀 vs 普通下单的根本差异：业务侧规则

维度	普通下单	秒杀
商品池	Product 主表	独立 SkillProduct 表
库存	与展示页同步	单独预热到 Redis，与主商品池隔离
用户预期	总能下单	大概率抢不到
重复点击	业务上是 bug	业务上是常态（用户疯狂点击）
失败的语义	”服务挂了”	”你手慢了”
限流策略	不挂限流（P0）	强制 SlidingWindow 3/s/用户
熔断策略	可熔断到熔断器返回降级	不熔断

为什么秒杀不熔断：熔断 = ”所有人都买不到”，业务上等价于**活动作废**——比”部分用户买不到”严重 10 倍。

为什么秒杀不挂 Idempotency：用户疯狂点击”再试一次”就是**业务预期**，幂等命中会让用户以为下单成功，反而误导。*internal/skill/service.go:144-152*；*internal/skill/routes.go:12-25* 秒杀路由

SlidingWindow Lua 与” 阈值 = 3” 的业务依据



业务侧选阈

值的依据:

- 真人最快 **300–500ms** 点一次，1 秒内 2–3 次封顶（除非用脚本）。
- 阈值 = 3 留 50% 余量给网络抖动 / 手抖连点 / 主动重试。
- 阈值 = 10 形同无限（一台脚本 1 秒打 50 次都过）。
- 阈值 = 1 客诉爆炸（用户网络稍差就被截）。

业务码 70001：限流命中，客服话术” 操作过于频繁，请稍后再试”，不是失败、不是错误，是” 稍等 1 秒重试就行”。`repository/cache/ratelimit.go:18–34 SlidingWindow Lua`；`pkg/e/code.go:57 ErrRateLimitExceeded=70001`

秒杀压测实测：30VU × 15s 误差 2.2%

场景：30 个并发用户用同一 **token** 打 /skill_product/skill，配置 Limit=3 Window=1s ByUser=true（复现”一个账号被脚本疯狂调用”的黑产画像）。

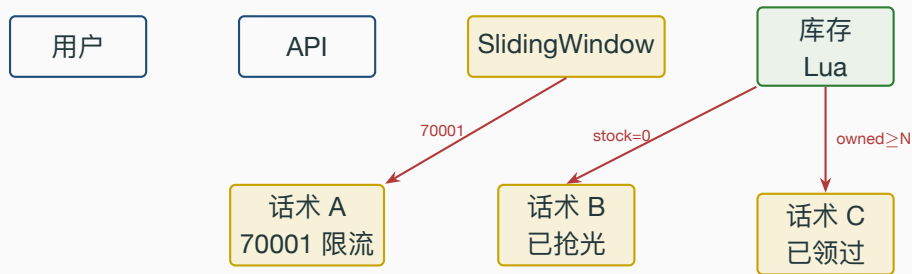
指标	实测	业务期望
通过请求数	46	$15s \times 3 = 45$
被限流数	781,624	黑产应全部拦下
误差	2.2%	< 5% 合格

业务侧解读：

- ”通过 46 次” $\approx$ 真用户最快也只能秒到 45 次/15s，业务合理。
- 被截 **78 万次**都是同一 **token** = 标准黑产画像。
- 误差 2.2% 对运营说”合规”，对 SRE 说”压力可控”，对客服说”真用户极少误伤”。

大促 SOP 联动：监控大盘看被限流数 / 通过数比值，如果飙升 = 黑产攻击，运营触发风控

秒杀客诉链路：从“我抢不到”到客服话术



客服必须区分三种“失败”：

- **70001 限流**：“操作过于频繁，请稍后再试”——1 秒后可重试。
- **库存 = 0 已抢光**：“已被其他用户抢走，欢迎下次参与”——终态、不可重试。
- **owned ≥ N 已领过**：“您已成功参与本次活动”——鼓励、不打击用户。

用户体验红线：绝不能说“系统繁忙”或“未知错误”——用户会以为是 bug，立刻投诉。

抢红包：公平性是用户体验生死线

抢红包的业务目的：留存而非促单

- **业务公式：**发 1000 个 1 元红包 → 1000 人参与 → 抢到的”开心”+ 没抢到的”下次还来”。钱花得不多，留存效果惊人。
- **核心差异：**和优惠券（促单）、秒杀（引流）不同——红包不直接 **GMV** 转化，目的是**每日打开率 / 7 日留存率**。
- **发包场景：**商家发（送老客）、平台发（节日大礼）、用户互发（社交裂变——gomall 当前是个人钱包发）。
- **业务关键点 1：**金额必须随机。如果每人 1 元 = 优惠券，不刺激；要有人抢到 5 元，有人抢到 1 分。
- **业务关键点 2：**总额必须精确。100 元发出去就是 100 元，多 1 分平台亏、少 1 分用户投诉。
- **业务关键点 3：**公平感。每份 ≥ 1 分 + 顺序打散 + 最后一份兜底——这是用户口碑的生死线。

公平性事故的真实代价：业务案例

反面教材：某红包活动 1000 元 / 100 份拆分时 bug 让第 1 个抢到的人独占 800 元，剩下 99 人抢 200 元（平均 2 元）。

现象	后果
社交媒体爆款	截图被疯传“XX 公司抢红包内定”，活动 ROI 直接归零
品牌信任崩塌	用户下次活动不参与：反正抢不到大头
监管关注	涉嫌诱导分享 + 不公平算法，运营部约谈
内部追责	算法工程师 + 产品 + QA 三方追责（这种 bug 压测覆盖不到）
真实修复成本	给所有参与者补发红包 + 公开道歉 + 算法外审

业务结论：抢红包的“公平感” $\neq$ “金额平均”——用户能接受抢到 1 分，但不能接受规则可疑。

gomall 用二倍均值法保证：(1) 每份 ≥ 1 分 (2) 总额守恒 (3) Shuffle 打散位置敏感。三件事任一缺失就是上述事故。repository/cache/redpacket.go:98-142 SplitRedPacket

二倍均值法：业务约束如何映射成代码

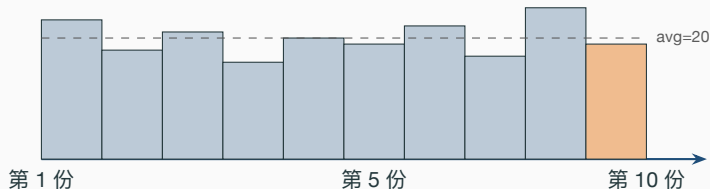
业务约束 → 算法选择：

- ”总额精确” → 最后一份 `out[count-1] = remain` 兜底吃剩。
- ”每份 ≥ 1 分” → 随机区间 `[1, hi]`, `maxThisRound = remain - (剩余份数)` 保证后续每份至少 1。
- ”随机有惊喜” → 上界 `2*avg-1`, 期望 = avg, 方差由 avg 控制。
- ”位置不敏感” → 最后 Shuffle 打散数组, 否则用户能推断”第一个抢的最大”。

Listing 6: *repository/cache/redpacket.go:107-140 SplitRedPacket* 主体

```
107 out := make([]int64, count)
108 remain := total
109 for i := 0; i < count-1; i++ {
110     left := int64(count - i)
111     avg := remain / left * 2 // [1, 2*avg-1] 区间上界
112     if avg < 2 { avg = 2 }
113     maxThisRound := remain - int64(count-1-i) // 留 1 分给后面每份
114     hi := avg - 1
115     if hi > maxThisRound { hi = maxThisRound }
```

200 元 / 10 份分布示意 + 用户感知



用户视角：抢到 25 元的觉得“运气真好”，抢到 16 元的觉得“还行”，抢到 0.01 元的会发朋友圈“手慢了哈哈”。

产品价值：每份不等 → 每个人都有故事讲 → 社交传播 → 下次活动自发参与。

对比反面：等额拆分 = 100 元 / 100 份 = 每人 1 元 → 没人晒图 → 留存效果归零。

repository/cache/redpacket.go:113-141；行业经验：拼手气红包 vs 等额红包的留存对比

抢红包 Lua：业务规则 8 行落地

业务规则三句话：(1) 一个人只能抢一次 (2) 抢完即终止 (3) 抢到金额是哪一份就是哪一份。

Listing 7: *repository/cache/redpacket.go:52-63 claimRedPacketScript*

```
52 if redis.call('HEXISTS', KEYS[2], ARGV[1]) == 1 then
53     return -2 -- 已领过
54 end
55 local amt = redis.call('LPOP', KEYS[1])
56 if not amt then
57     return -1 -- 已抢完
58 end
59 redis.call('HSET', KEYS[2], ARGV[1], amt)
60 redis.call('EXPIRE', KEYS[2], tonumber(ARGV[2]))
61 return tonumber(amt) -- 抢到金额
```

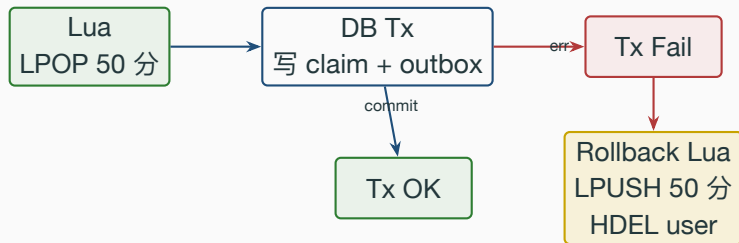
业务码翻译 (Go 侧 repository/cache/redpacket.go:168--190):

-1 → ErrRedPacketDrained → 客服话术” 红包已被抢完” (README 称作 RP-EMPTY 文案, 代码侧是 Go error)

-2 → ErrRedPacketAlreadyTaken → 客服话术” 您已领取过本红包”

> 0 → 200 + amount → 客户端展示动画

抢红包 Saga：资金链路的”最后一道防线”



业务事故场景：用户点抢，

Lua 已经 LPOP 了 50 分，但 DB 这一刻挂了——

- 不回滚：Redis 里这份金额”消失”，用户没收到钱 + 别人也抢不到这份 = 资金黑洞。
- 回滚：金额 LPUSH 回队头 + HDEL 撤销 user 标记 → 下一个用户能抢到 → 用户也可重试。

业务承诺：发出去多少钱，必须回到用户钱包。Saga 是这条业务承诺的代码兑现。

对应代码：rollbackRedPacketClaimScript（两个动作放一个 Lua 原子单元）。

repository/cache/redpacket.go:79-90 rollback; internal/redpacket/service.go:155-162 调用点

三个活动的横向业务对比

三种活动：业务目的 vs 代码骨架

维度	优惠券	秒杀	抢红包
业务目的	促单（推决策）	引流（拉新 / UV）	留存（日活 / 7 日）
资金方向	平台让利	平台亏本商品差价	个人钱包 / 营销预算
ROI 指标	核销率 / GMV	UV / 陪跑转化	日活 / 留存
用户预期	多领多得	大概率抢不到	金额随机但守恒
客服核心话术	” 已领过 / 已抢光”	” 已售罄 / 限流”	” 已抢完 / 已领过”
核心命令	DECR + INCR	ZADD + ZCARD	LPOP + HSET
数据结构	STRING × 2	ZSet	LIST + HASH
预热成本	一次 SET 库存	无（按需建窗口）	发包时全量预拆
回滚	INCR + DECR	无（限流不可逆）	LPUSH + HDEL

业务语义决定结构选型：金额同质 → 计数器；异质 → 队列；时间窗口 → ZSet。

反过来用就不行——红包用 DECR 算金额不守恒；优惠券用 LPOP 浪费；限流用 DECR 固定窗口边界 2× 突刺。

`repository/cache/coupon.go`, `ratelimit.go`, `redpacket.go` 三处实现

三段 Lua 的共同设计纪律：业务诉求驱动

1. **失败也要原子**：判断不通过什么都不写 → 防止”扣了库存但没记上单人”的资损。
2. **回滚也是 Lua**：DB 失败时补偿动作走同一种原子机制 → 防止 Go 侧两次 Redis 调用之间的崩溃。
3. **TTL 永远要设**：尤其领取标记类 key → 防止用户停留态 key 无限堆积撑爆 Redis。
4. **返回值留语义**：用 >0 / 0 / 负数区分 → 业务层 switch 翻译成客服话术。
5. **进程内 rand**：发包拆分时不用 `math/rand` 全局源 → 防止热点路径并发争锁。

业务侧意义：这 5 条纪律对应 5 类生产事故——

资损 / 双花 / OOM / 客服误导 / 高并发慢查。每一条都是真实事故烧出来的。

repository/cache/redpacket.go:92-96 splitRand 局部源；coupon.go:30-44 错误码语义

黑产对抗：业务后果与防线分层

黑产对营销 ROI 的真实破坏：业务后果帧

为什么营销是黑产首选：(1) 真金白银（券能转卖、红包能提现）(2) 规则透明必须对外通告 (3) 多账号便宜（手机号能买 8 分/号）。

黑产手法	业务后果
脚本扫券	万号自动领券 → 真用户领不到，运营预算打 7 折吃掉
机器人秒杀 撞库刷红包	1 秒打 50 次 → 真人 1 秒最多 2 次，真人 0% 抢到 用泄露密码盗号抢红包 → 用户钱包资金凭空消失， 投诉爆炸
券倒卖	黄牛领券后二手平台 50% 折价转卖 → 平台拉新归零

ROI 视角：上一帧算的 $ROI = 7.5$ ，是纯净用户假设。如果 30% 流量被黑产吃掉：
→ 核销率从 20% 跌到 14%（券给了脚本党 = 不会用）

gomall 拦的是哪一类黑产：能力分层

层级	工具	gomall 现状	能拦的黑产
L1 网络	Anti-DDoS / CDN 黑名单	未覆盖	大规模 DDoS / 已知恶意 IP
L2 网关	IP TokenBucket 100/IP	已接 (router.go:41 全局)	单机脚本 / 自动化爬虫
L3 业务	用户 SlidingWindow	秒杀 + 红包接, coupon 缺	单账号高频刷
L4 风控	设备指纹 / 行为模型	路线图	万号脚本 (须接外部风控)

每多一层，黑产成本翻倍：刷券既要换 IP (L2)、又要换号 (L3)、又要换设备指纹 (L4) ——单券薅羊毛就不划算了。

gomall 当前站住 L2 + L3：能挡掉 80% 业余黑产，专业黑产需要 L4 才够。

- **TokenBucket** 挡机器自动化：换 IP / 走代理才能绕，单 IP 100 QPS 拖住所有自动化。
- **SlidingWindow** 挡账号刷子：换账号才能绕，单号 3 次/秒让脚本撞不出经济性。
- 两层一起上，黑产必须同时凑齐 IP 池 + 账号池，门槛指数级抬高。

*routes/router.go:41 全局桶；internal/redpacket/routes.go:20-25 红包 SlidingWindow；
internal/skill/routes.go:17-24 秒杀 SlidingWindow*

coupon 路由的故意”漏洞”：业务取舍而非疏忽

现状：coupon/claim 只挂全局 TokenBucket，没挂用户 **SlidingWindow**。这是 bug 吗？

业务侧取舍表：

挂上 SlidingWindow	不挂
真用户多领被截（差体验）	真用户多领无限制（鼓励参与）
脚本党 5/min 撞限流停下	脚本党可继续刷（被 IP 桶慢拖）
Lua 已经管 PerUser	Lua 已经管 PerUser

决策：当前选**不挂**——信任 Lua 的 PerUser 上限 + IP 桶的”慢拖”，把体验留给真用户。

风险敞口：万号脚本党可以慢慢扫，目前靠”发券量预设 + 实时监控核销率”运营兜底（核销率 < 5% 触发活动暂停）。

未来路线图：接入风控雷达 L4 层后再追加 SlidingWindow（5 张/min/用户）。

客诉处理：从业务码到客服话术

业务真相：大促当天客服话务量是平时 5-10x，营销活动是投诉来源 TOP 3。

常见客诉拆解：

用户原话	业务码	客服话术 / 解决路径
” 我抢券失败了”	70001	” 操作过于频繁，请稍后再试” (1 秒后可重试)
” 我抢券失败了”	ErrCouponSoldOut	” 活动已结束 / 库存已抢完” (终态)
” 我抢券失败了”	ErrCouponPerUserExceed	” 每人限领 N 张，您已达上限” (鼓励性话术)
” 红包抢不到”	ErrRedPacketDrained	” 本轮红包已被抢完，下次再来” (终态)
” 红包金额很少”	业务正常	” 拼手气红包随机分配，下次试试手气” (解释机制)
” 秒杀页面卡死”	70001 / 网络	” 服务繁忙，请稍后再试” (前端降级文案)
” 秒杀没抢到”	业务正常	” 已被其他用户抢走，欢迎下次参与”
” 我抢到红包但用不了”	流程未完成	查 outbox 表 → 重投 red_packet.claimed 事件

`pkg/e/code.go:57-58 70001/70002`; `repository/cache/redpacket.go:23-26 Go error` 定义

客服话术红线：业务码到 UX 文案的映射

业务码 / 错误	用户看到的话术	绝不能说
70001 限流命中	”稍后再试”	”系统繁忙” / ”限流了”
70002 熔断打开	”支付服务暂忙，1 分钟后”	”下游挂了”
60002 幂等命中	”您的操作已受理”	”请勿重复点击”
50001 库存不足	”已售罄”	”缺货” / ”卖完了”（生硬）
ErrCouponSoldOut	”活动已结束”	”失败” / ”错误”
ErrRedPacketDrained	”红包已被抢完”	”没了” / ”结束”
ErrPerUserExceed	”每人限领 N 张”	”已超限”（冷冰）

话术哲学：

- **70001 不是失败**，是”稍后重试就行”——客户端可以**自动重试**（前端体验关键）。
- **Sold-out 是终态**，告诉用户活动已结束——**不要让用户重试**（浪费请求）。
- **鼓励性话术 > 错误性话术**——”已达上限”比”失败”友好 100 倍。

”我抢到红包但金额很少”：公平性解释 SOP

真实场景：用户抢到 1 分钱红包，截图发朋友圈”抠门平台”，客服收到投诉。

客服 SOP (gomall 推荐话术)：

1. **先共情：**”理解您看到金额比较小有点失望”。
2. **讲机制：**”拼手气红包采用**二倍均值算法**，每份金额随机分配，平均下来每份相同，但每个人抢到的金额会有差异”。
3. **讲事实：**”本次红包发出去的总金额是 `$total` 元，所有金额加起来恰好等于这个数——**平台没拿一分**”。
4. **讲公平：**”抢到大头和小头是**运气**，本次活动 `$count` 人中**每个人都有机会抢到最大份额**”。
5. **给补偿 (可选)：**发 5 元代金券安抚 (运营授权范围内)。

为什么必须有这套 SOP：上文”事故的真实代价”那帧——如果客服回”系统就这样”，用户会觉得”规则可疑”，社交媒体爆款风险。repository/cache/redpacket.go:98-142 二倍均值实现是这套

运营 SOP：活动准备 + 盘点

大促前 7 天准备清单：5 角色协同 i

角色	T-7 准备项
运营	活动规则 review / 券面值与数量定 / 红包总额预算 / 预热券分批投放
SRE	Redis 库存 预热 （活动券 / 秒杀商品 / 红包预拆 SET 入库） / 限流阈值 预设 到大促值（3/s → 评估是否拉到 5/s） / 监控大盘 重点指标 ：70001 数 / Lua RTT p99 / Redis CPU
客服	话术下发 （上一节那张映射表） / 客诉演练彩排 / 常见 FAQ 上线
风控	黑名单池 预热 （已知黑产 IP / 账号） / 风控规则压测
商家（路线图）	活动页 素材审核 / 库存 预占 / 客服承接预案

T-1 复盘：压测 30VU × 15s 跑一遍验证限流和库存预热——看是否 46/45 误差 < 5%。

T-0 0 点联动：

- 运营在战情室看 GMV 实时数。
- SRE 看监控大盘，准备熔断 / 降级开关。
- 客服**增援 5x** 人手。

- 风控实时识别单账号 / 单 IP 异常 → 触发临时拉黑。

stressTest/REPORT.md §5; README ”运行” / docker-up / make env-up 准备依赖

活动结束盘点：4 个维度复盘

维度	复盘项 + 数据来源
实际 ROI	发券数 / 领取率 / 核销率 / 带动 GMV / 让利金额 (coupon_batch + user_coupon JOIN orders)
黑产识别	70001 限流次数 / 单账号高频参与排行 / 核销率异常低的批次 (< 5% 即可疑)
用户投诉	投诉 / 客服工单分类 (限流误伤 / 抢不到 / 金额疑问) + 平台口碑监控
系统稳定	Lua RTT p99 / Redis CPU 峰值 / 超时熔断次数 / 误差率

产出物 (运营平台路线图):

- 活动复盘报告: 4 维度数据 + 下次改进项。
- 黑名单沉淀: 识别出的黑产账号 / IP 进入永久黑名单。
- 话术迭代: 客诉热点 → 下次话术增补。
- 阈值校准: 本次 SlidingWindow 限流数 / 通过数 → 下次活动参考。

gomall 现状: 复盘数据靠 stressTest/REPORT.md 沉淀, 运营平台暂未做 (路线图)。README " 技术覆盖 " 营销活动表; docs/architecture/feature-matrix.md 运营平台

各角色看到的关键指标

三个活动的各角色视图汇总

角色	关心指标	数据来源
运营	领取率 / 核销率 / GMV / 拉新数 / ROI	coupon_batch.finished + 订单表 JOIN
风控	70001 占比 / 单账号高频 / 核销率异常	业务码统计 + 用户行为日志
客服	投诉单 → 业务码反查 / 话术覆盖率	status 字段 + Jaeger trace 反查
SRE	Lua RTT p99 / Redis CPU / 限流通过率	Prometheus / Jaeger
财务	红包总额守恒 / 让利预算 vs 实际	DB sum vs outbox events
C 端用户	抢到了吗 / 公平吗 / 能用吗	用户卡包 + 抢到记录

业务关键差异：

- 秒杀可用率最低（99%）——”全场抢光”本就是营销目标，5xx 短抖动不算事故。
- 抢红包反过来——抢到没入账 = 投诉，所以 Saga 必须兜底。
- **70001** 比例上升：运营看的是”参与热度”，风控看的是”被刷”，同一个数字两种解读。

对照 *deck 10 P0-P3* 等级表；*pkg/e/code.go* 业务码

大促当天监控大盘的 6 个核心图



6 个图，3 类预警：

- **业务预警：** GMV 增长曲线 vs 预期 → 偏离 > 20% 运营介入。
- **安全预警：** 70001 飙升 + 核销率骤降 → 黑产攻击，风控降级。
- **系统预警：** Redis CPU > 80% 或 Lua p99 > 100ms → SRE 准备熔断。

联动开关：监控大盘旁边常备 3 个红色按钮——

1. **临时拉高限流** (3/s → 1/s) ——黑产攻击降级。
2. **暂停活动** (运营授权) ——异常事故止血。

Q&A

Q&A (上): 业务取舍与原子性

Q1: 优惠券为什么不挂用户 SlidingWindow?

A: 业务侧**鼓励多领** (沉没成本 $\rightarrow$ 转化), Lua 已管 PerUser; 信任 IP 桶 + 运营核销率监控兜底。

代码: routes/router.go:41; internal/coupon/routes.go:10--13;
repository/cache/coupon.go:37。

Q2: 100 万发券的 ROI 怎么算?

A: 发券 $\rightarrow$ 领取率 60% $\rightarrow$ 核销率 20% $\rightarrow$ 带动 GMV。让利 12 万换 90 万 GMV, $ROI = 7.5$ 。黑产吃 30% 流量 $\rightarrow$ ROI 跌到 3.0。

代码: internal/coupon/service.go:62--96 Claim (gomall 暂未实装 ROI 仪表盘)。

Q3: 抢红包 Lua 已经 LPOP 了, DB 落库失败怎么办?

A: Saga 回滚——rollbackRedPacketClaimScript, LPUSH 金额回队头 + HDEL 撤销 claimed 标记。**发出去多少必须回来是业务承诺。**

代码: repository/cache/redpacket.go:79--90、
internal/redpacket/service.go:155--162。

Q&A (下): 黑产、客服与公平性

Q5: 3/s/用户这个阈值是怎么定的?

A: 真人 300-500ms 点一次, 3 留 50% 余量; 压测 46/45 误差 2.2%; 取 1 客诉爆, 取 10 形同没限。

代码: `internal/skill/routes.go:17--24`; `stressTest/REPORT.md §5`。

Q6: 用户抢到 1 分钱红包投诉, 客服怎么回?

A: 5 步 SOP —— 共情 / 讲机制 (二倍均值法) / 讲事实 (总额守恒) / 讲公平 (运气) / 给补偿 (可选)。绝不能说“系统就这样”。

代码: `repository/cache/redpacket.go:98--142` 是这套话术的代码兑现。

Q7: 黑产用一万个账号刷 coupon, gomall 能挡住吗?

A: 当前挡不住——只能靠 IP 桶 100 RPS/IP 拖速度 + 运营核销率监控 (< 5% 暂停)。下一步追加 SlidingWindow + 接外部风控 (L4)。

代码: `routes/router.go:41`; `internal/coupon/routes.go:10--13`。

Q8: 抢券失败、抢红包失败、秒杀失败, 客服怎么区分?

A: 按业务码: 70001 跟读 “稍后再试”; 70002 “已抢完”; 70003 “已领过”

代码位置一览（一）：业务实现

coupon Lua（原子扣减 + 单人限领） *repository/cache/coupon.go:30-44*

coupon 错误码翻译 *repository/cache/coupon.go:56-85*

coupon 回滚 Lua（原子归还 + 返回 error） *repository/cache/coupon.go:50-54、:87-93*

coupon 业务层 Claim *internal/coupon/service.go:62-96*

coupon 库存预热（创建批次） *internal/coupon/service.go:27-54*

coupon DB 锁路径（clause.Locking + 随机券码） *internal/coupon/repo.go:44-90、:127-137*

coupon 路由（无 SlidingWindow，业务取舍） *internal/coupon/routes.go:10-13*

红包二倍均值法拆分 *repository/cache/redpacket.go:98-142 SplitRedPacket*

红包预拆 RPU SH Lua *repository/cache/redpacket.go:34-43*

红包抢取 Lua（HEXISTS + LPOP + HSET） *repository/cache/redpacket.go:52-63*

红包 Saga 回滚 Lua *repository/cache/redpacket.go:79-90*

红包 Service 调用 Saga 点 *internal/redpacket/service.go:155-162*

红包过期回收 *repository/cache/redpacket.go:65-77 ReleaseRedPacketLeft*

红包路由（SlidingWindow + Idempotency） *internal/redpacket/routes.go:17-27*

秒杀业务（详情/下单读单品 key + 回源回填） *internal/skill/service.go:75-103、:144-152*

秒杀路由（SlidingWindow 3/s/用户） *internal/skill/routes.go:17-24*

代码位置一览（二）：限流、业务码、压测

SlidingWindow Lua (ZSet 滑窗) *repository/cache/ratelimit.go:18-34*

全局 TokenBucket (100 RPS / IP) *routes/router.go:41 middleware/ratelimit.go*

业务码 70001 / 70002 / 60002 / 50001 *pkg/e/code.go:49-58*

Go error (红包 / 优惠券) *repository/cache/coupon.go:56-59; redpacket.go:22-26*

压测：秒杀 + 滑动窗口 (46/45 误差 2.2%) *stressTest/REPORT.md §5; seckill_rate_limit.js*

压测：优惠券 Redis Lua vs DB (51K / 50K RPS) *stressTest/REPORT.md 链路压测表第 5-6 行*

拼手气红包模型 (gomall 已实现) *internal/redpacket/service.go:41-175 Create + Claim*

业务边界 (未做项) *README "业务边界" + docs/architecture/feature-matrix.md*

配套技术 deck: 07-inventory (库存防超发) / 10-traffic-governance (限流熔断细节) / 11-consistency (Outbox 与 Saga) / 12-merchant-ops (监控大盘)